Practical 2

Classes, Objects, Constructors & Encapsulation

Part A — Practice Programs (Hour 1)

1) Smart Thermostat — constructor chaining, static counter, MIN/MAX constants

Code

```
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        this.temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX) temperature++;
        else System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN) temperature--;
        else System.out.println("Already at minimum (16)");
    }

    public int getTemperature() { return temperature; }
    public static int getActiveCount() { return activeCount; }

    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Hall", 25);
        Thermostat t2 = new Thermostat("Bedroom");
        System.out.println("--- " + t1.location + " (start " + t1.getTemperature() + ") raise x10
            ---");
        for (int i = 0; i < 10; i++) {
            t1.raise();
            System.out.println("Temp: " + t1.getTemperature());
        }
        System.out.println("--- " + t1.location + " lower x20 ---");
        for (int i = 0; i < 20; i++) {
            t1.lower();
            System.out.println("Temp: " + t1.getTemperature());
        }
        System.out.println(t2.location + " starts at " + t2.getTemperature());
        System.out.println("Active thermostats: " + getActiveCount());
    }
}
```

Output

```
--- Hall (start 25) raise x10 ---
Temp: 26
Temp: 27
Temp: 28
Temp: 29
Temp: 30
Already at maximum (30)
Temp: 30
Already at maximum (30)
Temp: 30
Already at maximum (30)
Temp: 30
Already at maximum (30)
Temp: 30
Already at maximum (30)
Temp: 30
--- Hall lower x20 ---
Temp: 29
Temp: 28
Temp: 27
Temp: 26
Temp: 25
Temp: 24
Temp: 23
Temp: 22
Temp: 21
Temp: 20
Temp: 19
Temp: 18
Temp: 17
Temp: 16
Already at minimum (16)
Temp: 16
Already at minimum (16)
Temp: 16
Already at minimum (16)
Temp: 16
Already at minimum (16)
Temp: 16
Already at minimum (16)
Temp: 16
Already at minimum (16)
Temp: 16
Bedroom starts at 22
Active thermostats: 2
```

2) Cinema Show — final capacity, book/cancel, static totalBooked

Code

```
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        seatsAvailable = Math.min(capacity, seatsAvailable + n);
    }

    public int getSeatsAvailable() { return seatsAvailable; }
    public static int getTotalBooked() { return totalBooked; }

    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Interstellar", 50);
        System.out.println("Show: " + show.title + ", capacity " + show.capacity);
        System.out.println("book(20) -> " + show.book(20) + ", seats = " +
            show.getSeatsAvailable());
        System.out.println("book(25) -> " + show.book(25) + ", seats = " +
            show.getSeatsAvailable());
        System.out.println("book(10) -> " + show.book(10) + ", seats = " +
            show.getSeatsAvailable());
        show.cancel(15);
        System.out.println("cancel(15), seats = " + show.getSeatsAvailable());
        show.cancel(100);
        System.out.println("cancel(100), seats = " + show.getSeatsAvailable() + " (capped at
            capacity)");
        System.out.println("Total booked: " + getTotalBooked());
    }
}
```

Output

```
Show: Interstellar, capacity 50
book(20) -> true, seats = 30
book(25) -> true, seats = 5
book(10) -> false, seats = 5
cancel(15), seats = 20
cancel(100), seats = 50 (capped at capacity)
Total booked: 45
```

3) Parking Lot (For Advanced Learners — home) — capacity checks, static revenue

Code

```
public class ParkingLot {
    private int twoWheelers, fourWheelers;
    private final int twoCap, fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
    }

    public void park(String type) {
        if (type.equals("two") && twoWheelers < twoCap) {
            twoWheelers++;
            revenue += 20;
            System.out.println("Parked two-wheeler (" + twoWheelers + "/" + twoCap + ")");
        } else if (type.equals("four") && fourWheelers < fourCap) {
            fourWheelers++;
            revenue += 40;
            System.out.println("Parked four-wheeler (" + fourWheelers + "/" + fourCap + ")");
        } else {
            System.out.println("Full - " + type + " rejected");
        }
    }

    public void leave(String type) {
        if (type.equals("two") && twoWheelers > 0) twoWheelers--;
        else if (type.equals("four") && fourWheelers > 0) fourWheelers--;
        System.out.println(type + " left");
    }

    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 1);
        lot.park("two");
        lot.park("two");
        lot.park("two");
        lot.park("four");
        lot.park("four");
        lot.leave("two");
        lot.park("two");
        lot.leave("four");
        lot.leave("four");
        System.out.println("Occupancy: two=" + lot.twoWheelers + ", four=" + lot.fourWheelers);
        System.out.println("Revenue: " + revenue);
    }
}
```

Output

```
Parked two-wheeler (1/2)
Parked two-wheeler (2/2)
Full - two rejected
Parked four-wheeler (1/1)
Full - four rejected
two left
Parked two-wheeler (2/2)
four left
four left
Occupancy: two=2, four=0
Revenue: 100
```

Part B — Project Work (Hour 2): MiniBank

Customer and Account are the foundation classes of MiniBank. The balance can change only through deposit() and withdraw(); there is no public setter. Ids are generated from private static counters.

1) Customer — private fields, final customerId, static counter and generateCustomerId()

Code

```
public class Customer {
    private final String customerId;
    private String name;
    private String email;
    private String mobile;
    private static long customerCounter = 100;

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public Customer(String name, String email, String mobile) {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
}
```

2-4) Account — final accountNumber, two constructors (this(ownerName, 0)), deposit and withdraw

Code

```
public class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static int counter = 0;

    public Account(String ownerName, long openingBalance) {
        counter++;
        this.accountNumber = String.format("AC%04d", counter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public void deposit(long amount) {
        if (amount <= 0) return;           // supplementary: reject negative amounts
        balance += amount;
    }

    public boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    // supplementary: move money using withdraw + deposit
    public static boolean transfer(Account from, Account to, long amount) {
        if (from.withdraw(amount)) {
            to.deposit(amount);
            return true;
        }
        return false;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

Supplementary problems covered: transfer() helper using withdraw + deposit, and negative amounts are rejected without changing the balance.

5) main — Account[] array, deposits and withdrawals

Code

```
public class MiniBank {
    public static void main(String[] args) {
        Customer c1 = new Customer("Riya", "riya@example.com", "9876543210");
        Customer c2 = new Customer("Aman", "aman@example.com", "9123456780");
        System.out.println("Customers: " + c1.getCustomerId() + " (" + c1.getName() + "), "
                + c2.getCustomerId() + " (" + c2.getName() + ")");

        Account[] accounts = {
            new Account("Riya", 5000),
            new Account("Aman", 2000),
            new Account("Neha")
        };

        System.out.println("--- Spec scenario on " + accounts[0].getAccountNumber() + " ---");
        System.out.println("Opening balance: " + accounts[0].getBalance());
        accounts[0].deposit(2000);
        System.out.println("After deposit(2000): " + accounts[0].getBalance());
        System.out.println("withdraw(3000) -> " + accounts[0].withdraw(3000) + ", balance " +
            accounts[0].getBalance());
        System.out.println("withdraw(10000) -> " + accounts[0].withdraw(10000) + ", balance " +
            accounts[0].getBalance());

        System.out.println("--- Other operations ---");
        accounts[1].deposit(500);
        accounts[2].deposit(-100);
        System.out.println("deposit(-100) on " + accounts[2].getAccountNumber() + " ignored,
            balance " + accounts[2].getBalance());
        System.out.println("transfer 1000 AC0001 -> AC0003: " + Account.transfer(accounts[0],
            accounts[2], 1000));

        System.out.println("--- Final balances ---");
        for (Account a : accounts) {
            System.out.println(a.getAccountNumber() + " | " + a.getOwnerName() + " | " +
                a.getBalance() + " | active=" + a.isActive());
        }
    }
}
```

Output

```
Customers: CUST101 (Riya), CUST102 (Aman)
--- Spec scenario on AC0001 ---
Opening balance: 5000
After deposit(2000): 7000
withdraw(3000) -> true, balance 4000
withdraw(10000) -> false, balance 4000
--- Other operations ---
deposit(-100) on AC0003 ignored, balance 0
transfer 1000 AC0001 -> AC0003: true
--- Final balances ---
AC0001 | Riya | 3000 | active=true
AC0002 | Aman | 2500 | active=true
AC0003 | Neha | 1000 | active=true
```

Package layout, build and tag

Code

```
minibank/
 |-- Account.java
 |-- Customer.java
 +-- MiniBank.java

javac *.java
java MiniBank
cd ..
git add .
git commit -m "Practical 2 - classes and encapsulation"
git tag lab-02
```

Balance 5000 -> 7000 after deposit(2000) -> 4000 after withdraw(3000); withdraw(10000) returns false and the balance stays 4000. The balance cannot be modified from outside the class because the field is private and has no setter.

Key Questions / Analysis / Interpretation

1. What is encapsulation, and how do private fields with public getters enforce it?

Encapsulation bundles data and the methods that work on it inside one class and hides the data from outside code. Fields are declared private, so other classes cannot read or write them directly; they must use public methods.

Getters give read-only access, and the only ways to change the balance are deposit() and withdraw(), which check every change. That keeps the object always in a valid state (for example, the balance can never be set to a

negative number by outside code).

2. Why is accountNumber declared final?

An account number identifies the account for its whole life. final means it can be assigned once (in the constructor) and never changed, so it cannot be overwritten by mistake, and equals/hashCode or map keys built on it stay stable.

3. What is the difference between a static field and an instance field when generating ids?

An instance field belongs to each object, so every Account has its own copy. A static field belongs to the class and there is one shared copy. The id counter must be static so every new account increments the same number and gets a different id (AC0001, AC0002, ...). If the counter were an instance field, each object would start again from zero and ids would repeat.